CI/CD Research and Automated Testing – Team Wack BucHunt

1. Continuous Integration and Continuous Deployment (CI/CD)

Our team researched implementing a CI/CD pipeline using GitHub Actions, GitHub Container Registry (GHCR), and Docker Compose to automate the testing, building, and deployment of BucHunt.

The proposed process is:

1. A developer merges code into the `sprint4` branch.
2. GitHub Actions automatically runs tests and builds the Docker images.
3. If the tests and builds succeed, GitHub Actions publishes the new images to GHCR.
4. A scheduled deployment script on our Linux server checks GHCR for updated images.
5. If newer images are available, the server downloads them.
6. Docker Compose recreates the affected containers using the new images.
7. The updated BucHunt application becomes available without requiring manual deployment.

The Linux server initiates outbound connections to retrieve the images, eliminating the need to expose SSH publicly.

Example Deployment Timeline

Time	Event
12:00	Developer merges code into `sprint4`
12:01	GitHub Actions builds the Docker image
12:02	GitHub Actions publishes the image to GHCR
12:03	Linux deployment script downloads the updated image
12:03	Docker Compose recreates the affected container
12:04	Updated BucHunt application is available

These times are illustrative. Actual deployment times depend on build duration, image size, and server availability.

Benefits

- Eliminates manual rebuilding and deployment.
- Automatically runs tests before publishing new images.
- Keeps Docker builds off the production server.
- Allows deployments without exposing SSH to the internet.
- Provides a repeatable deployment process and a record of build results through GitHub Actions.

2. Existing Automated Backend Tests

We also reviewed an existing automated integration test for the BucHunt backend. The test uses Node.js's built-in testing framework to verify that hunt tasks are correctly retrieved from CSV files through the backend API.

How the Test Works

1. Creates a temporary directory for test data.
2. Finds an available network port.
3. Starts a separate instance of the BucHunt backend (`index.js`) in test mode.
4. Waits for the backend to respond.
5. Creates sample hunt data and sends HTTP requests.
6. Compares the API responses against expected results.
7. Stops the test server and removes the temporary files.

Existing Test Cases

Test	Purpose	Expected Result
Missing Tasks	Request tasks for a hunt that does not exist	HTTP 404 with `"Hunt not found."`
Empty Tasks	Create a hunt containing zero tasks	HTTP 200 with an empty task array
Ten Tasks	Create a hunt containing 10 tasks	HTTP 200 with 10 tasks and unique sequential IDs
Hundred Tasks	Create a hunt containing 100 tasks	HTTP 200 with 100 tasks, unique sequential IDs, and no exposed answers

The fourth test also checks that the API does not return the correct task answers to players. This is important because players should not be able to discover answers by inspecting API responses.

Limitations of Existing Tests

The current tests do not verify:

- Correct or incorrect answer submissions.
- Multiple players joining the same hunt.
- Tracking completed tasks.
- Frontend functionality.
- Docker builds or deployment.
- Data persistence after a server restart.

Additionally, the tests create CSV files directly rather than creating hunts through the application's API. Therefore, they verify task retrieval but not the complete hunt creation process.

3. Integrating Tests into CI/CD

The existing backend tests could be incorporated into GitHub Actions as part of the CI/CD pipeline.

When new code is pushed, GitHub Actions would run the automated tests before building and publishing updated Docker images.

If the tests pass, the workflow continues with image creation and publication to GHCR. If any tests fail, the workflow stops and prevents the failing version from being published through that workflow.

This approach helps prevent changes that break existing functionality from reaching the deployed application.

4. Conclusion

Our research identified GitHub Actions, GHCR, and Docker Compose as a potential solution for automating BucHunt deployments.

We also reviewed existing backend integration tests that could support continuous integration by validating task retrieval and API responses.

Combining automated testing with containerized deployment would reduce manual work, improve deployment consistency, and help the team identify problems before new versions are released.

The proposed CI/CD pipeline remains a researched implementation approach rather than a confirmed completed deployment.

- Reduces manual deployment steps
- Each step in the deployment process can be executed inside of a specified container on a worker, not directly on the worker itself.
- Keeps worker thin
- All you need is Docker
- Octopus Cloud has docker installed
- No need for extra worker configure
- The worker is the machine or server.

Octopus Deploy + GitHub: A Beginner's Guide

A plain-English overview and set of how-tos for deploying software with Octopus Deploy and GitHub Actions. No CI/CD experience needed.

1. The problem this solves

Software is written by developers, but it has to end up running on a server for people to use it. Doing that by hand (copying files, editing settings, restarting things) is slow and error-prone.

- **CI (Continuous Integration):** each time a developer saves a change, a tool automatically builds the software and runs tests. The output is a **package**, a bundle of files ready to install.
- **CD (Continuous Delivery):** a second tool takes that package and installs it on your servers, step by step, the same way every time.

GitHub Actions handles the CI half. **Octopus Deploy** handles the CD half. Octopus doesn't build your code; it focuses on releasing and deploying it well.

2. Key terms

- **Project:** one application you deploy (for example, "Shopping Cart API").
- **Environment:** a place software runs, such as Dev, Test, or Production.
- **Deployment process:** the checklist of steps that installs the app (stop the service, copy files, restart). Octopus never skips a step or runs them out of order.
- **Release:** a frozen snapshot of your app version plus its deployment steps. The same release goes to Test and then Production, so what you tested is what ships.
- **Variables:** settings that differ per environment, like a database address. They can be scoped to specific environments, so nobody edits config files by hand.
- **Lifecycle:** the promotion rules, such as "a release must go through Test before Production."
- **Manual intervention:** an approval step. It can be required for Production but skipped for Dev.
- **Tentacle:** a small agent installed on a server so Octopus can run deployment steps there.
- **Runbook:** automation for routine tasks that aren't deployments, such as restarting services.

3. The flow at a glance

1. A developer pushes code to GitHub.
2. GitHub Actions builds and tests it, then packages it.
3. The package is pushed to Octopus.
4. Octopus creates a release and deploys it to Dev.
5. The release is promoted to Test, then (after approval) to Production, using rules set in Octopus.
6. The Octopus dashboard shows which version is running where.

4. How-to: set up Octopus

Menu names can shift slightly between Octopus versions. If something looks different, check the docs linked at the end.

Create an environment

1. In the Octopus web portal, go to **Infrastructure** → **Environments**.
2. Click **Add Environment**.
3. Name it (for example, *Development*) and click **Save**.
4. Repeat for *Test* and *Production*.

Create a project

1. Go to **Projects** → **Add Project**.
2. Give it a name that matches your app (for example, *MyApp*).
3. Keep the default lifecycle for now. You can tighten it later.

Add a deployment process

1. Open your project and choose **Deployment Process**.
2. Click **Add Step** and pick a step type that matches your app (for example, deploy a package to a server, a website, or a cloud service).
3. Choose which environments or targets the step runs on.
4. Save. Start with one simple step and add more as you go.

Set per-environment settings with variables

1. In your project, open **Variables**.
2. Add a variable, such as `DatabaseServer`.
3. Give it a different value for each environment by scoping each value to Development, Test, or Production.
4. Reference the variable in your deployment step instead of typing the value in.

Require approval before Production

1. In the deployment process, add a **Manual Intervention** step.
2. Scope it to the Production environment only, so Dev and Test skip it.
3. Choose who is allowed to approve.

Roll back

Roll back by redeploying an earlier release to the environment. Because releases are frozen snapshots, you get exactly what ran before.

5. How-to: connect GitHub to Octopus

Create an API key

An API key lets GitHub talk to Octopus, like a password for a robot.

1. In Octopus, open your user profile and choose **My API Keys**.
2. Create a new key with a purpose such as "GitHub Actions" and copy it. You usually can't view it again later.

Store secrets in GitHub

1. In your GitHub repo, go to **Settings** → **Secrets and variables** → **Actions**.
2. Click **New repository secret**.
3. Add `OCTOPUS_API_KEY` (the key you copied) and `OCTOPUS_URL` (your Octopus address).
4. Never paste these values directly into the workflow file.

Add the workflow file

Create `.github/workflows/deploy.yml` in your repo. This is a sketch based on Octopus's v3 examples, so adjust names to match your project:

```
name: Build and deploy
on: [push]

env:
  OCTOPUS_API_KEY: ${{ secrets.OCTOPUS_API_KEY }}
  OCTOPUS_URL: ${{ secrets.OCTOPUS_URL }}
  OCTOPUS_SPACE: Default

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      # ...your build and test steps here...

      - uses: OctopusDeploy/create-zip-package-action@v3
        id: package
        with:
          package_id: MyApp
          version: 1.0.${{ github.run_number }}
          output_folder: packaged
          base_path: build
          files: "**/*"

      - uses: OctopusDeploy/push-package-action@v3
        with:
          packages: ${{ steps.package.outputs.package_file_path }}

      - uses: OctopusDeploy/create-release-action@v3
        id: release
        with:
          project: MyApp
          package_version: 1.0.${{ github.run_number }}

      - uses: OctopusDeploy/deploy-release-action@v3
        with:
          project: MyApp
          release_number: ${{ steps.release.outputs.release_number }}
          environments: Development
```

What each action does

- **create-zip-package-action:** bundles your build output into a package.
- **push-package-action:** uploads the package to Octopus.
- **create-release-action:** creates a release in Octopus.
- **deploy-release-action:** deploys the release to the environments you name.
- **push-build-information-action** (optional): sends commit and build details so Octopus can show what changed.
- **await-task-action** (optional): makes GitHub wait for the deployment to finish.

Run it and check the result

1. Commit the workflow file and push to GitHub.
2. Open the **Actions** tab in your repo to watch the build run.
3. Open the Octopus dashboard. A new release should appear and deploy to Development.
4. In Octopus, promote the release to Test, then Production, when you're ready.

6. Things to know

- GitHub-hosted runners must be able to reach your Octopus Server over the internet. If Octopus is on a private network, you need self-hosted runners. Octopus Cloud avoids this problem.
- You can pin an action to a major, minor, or exact version. Using the major version (`@v3`) is a reasonable default.
- Let GitHub own building and testing, and let Octopus own promotion, approvals, and environment-specific settings.
- Start small: one app, two environments, one deployment step.

7. Sources

- Octopus getting started overview: https://octopus.com/docs/getting-started
- Octopus glossary: https://octopus.com/docs/getting-started/glossary
- Octopus deployments concepts: https://octopus.com/docs/deployments
- Octopus environments: https://octopus.com/docs/infrastructure/environments
- v3 GitHub Actions announcement, with a full example: https://octopus.com/blog/github-actions-for-octopus-deploy-v3
- Create-release action (inputs and options): https://github.com/OctopusDeploy/create-release-action
- Octopus GitHub Actions integration guide: https://oc.to/github-docs
- Version pinning and build information: https://octopus.com/blog/new-in-github-actions

TESTING NOTES

Nathaniel:

We do not HAVE to use octopus in github, we can manually write tests in a file for our project.

You define a workflow in `.github/workflows/<name>.yml`.

```
name: CI Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '18'
      - run: npm install
      - run: npm test
```

Alex:

1. What is automated unit testing?

Automated unit testing involves testing individual units of source code, such as:

- ***Methods***
- ***Functions***
- ***Classes***
- ***Individual components***

The purpose is to identify errors early and ensure that individual pieces of software behave as expected.

2. Create a GitHub Actions workflow

A GitHub Actions workflow is a configuration file that tells GitHub what tasks to perform.

To create one:

1. *Open your GitHub repository.*
2. *Navigate to the repository's main page.*
3. *Click Add file → Create new file.*
4. *Name the file: .github/workflows/tests.yml*

3. Configure the automated tests

Let's use Python and pytest as a general example.

Suppose you already have automated tests in a file called `test_calculator.py`.

Your GitHub workflow could look like this:

None

```
name: Automated Unit Tests

on:

  push:

    branches: [main]

  pull_request:

    branches: [main]

jobs:

  test:
```

```
    runs-on: ubuntu-latest
    steps:
      - name: Get repository code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Install dependencies
        run: |
          python -m pip install pytest
      - name: Run automated tests
        run: pytest
```

Important: This is an example for Python. Other programming languages require different setup and test commands.

4. Commit and push the workflow

Once you've created the workflow file:

1. ***Click Commit changes.***
2. ***Save the file to your repository.***
3. ***Push your code if you're working locally.***

GitHub will recognize the workflow and execute it when one of the configured events occurs.

5. View your automated test results

After pushing your code:

1. *Open your GitHub repository.*
2. *Click the Actions tab.*
3. *Select the workflow named Automated Unit Tests.*
4. *Click the most recent workflow execution.*
5. *Review the results.*